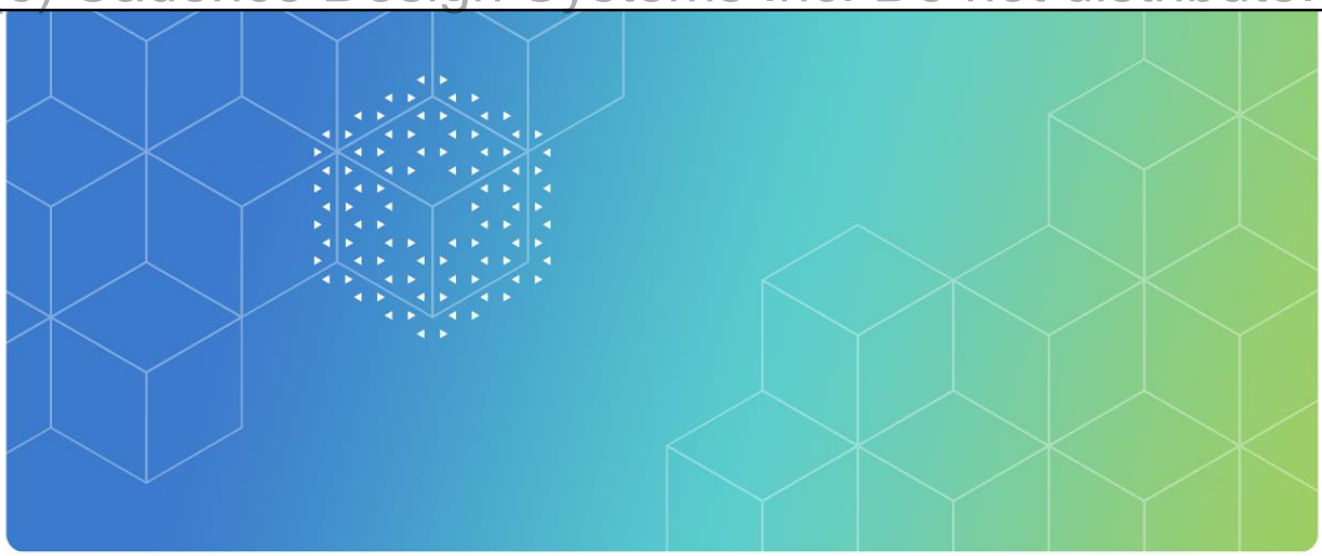


Module 8.1

Finite State Machines (FSMs)

cadence

This page does not contain notes.

Objective

In this module, you will:

- Define Finite State Machine (FSM) and different ways of implementation.

Topics include:

- FSM basics
- Types of FSM
- FSM structure
- State encoding

This submodule will look at Finite State Machines (FSMs) and different implementations. We will look at FSM basics, different types of FSMs, and optimization of the FSM structure and state encoding.

What Is a Finite State Machine?

An FSM is a control path.

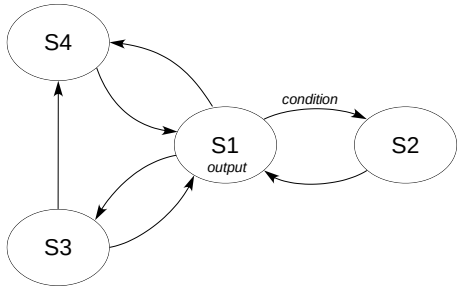
- It is clocked, i.e., sequential.
- It has a number of defined “states” that it can be in.
- Transition between states are determined by the combination of the current state and the current inputs to the FSM at a clock edge.
- Outputs from the FSM can be a function of:
  - Current state only (Moore)
  - Current state and the current inputs (Mealy)

cadence

An FSM is predominantly used to design sequential controllers and control paths. It is a clocked block with an internal state chosen from a defined list of allowed values. The FSM transitions from one state to another based on the current state and the inputs. The FSM outputs are defined by the current state only in a Moore FSM or the current state and the inputs for a Mealy FSM, as we shall see.

Moore FSM Representation

- S1 . . . S4 are the states of the FSM.
- Directed arrows are allowed transitions between states.
- *condition* is the input condition that must be satisfied to undergo a specified state transition.
- *Output* is the set of outputs associated with the state.
  - Requires a separate state for each combination of outputs.
- Transitions occur on the clock edge.



FSM can only be in one of the defined states.

In a Moore FSM, the outputs are functionally dependent only on the current state.

- Hence output values are allied to states.

The diagram shows only one condition and output.

In reality, all states will have an associated set of outputs, and all transitions should be labeled with a transition.

- Unlabelled transitions are automatically taken at the next clock, i.e., unconditionally.

An FSM is defined using a state diagram. This defines the allowed states of the FSM, S1 to S4. Arrows define the allowed transitions between states, and the transitions can be dependent on an input condition. The output values for each state are defined, and in a Moore FSM, the outputs are a condition of the state only. Therefore, we need a separate state for each unique combination of outputs. Remember, state transitions are synchronous to the clock edge. Although this diagram shows only one condition and output, in reality, all states will have an output, and all transitions should be labeled with a condition unless they are truly unconditional.

Mealy FSM Representation

Similar representation except …

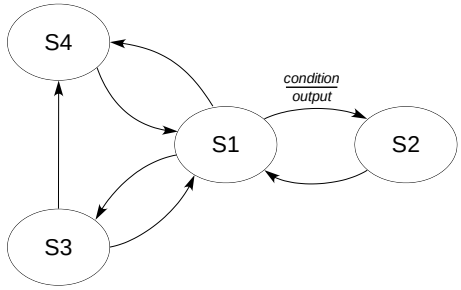
Associated with each transition is:

$$\frac{\textit{condition}}{\textit{output}}$$

condition defines when the transition occurs.

output is the set of outputs associated with the *inputs and state.*

- Each state can have different output combinations.
- Fewer states are required!



5

cadence

In a Mealy FSM, the outputs are functionally dependent on the current state and inputs.

Condition and output are associated with the transition.

- Output is dependent on the input (defines condition) and the state (where the arrow is directed to).

A Mealy state diagram is very similar to a Moore. However, as the outputs in a Mealy depend on the inputs as well as the current state, the transition condition is also associated with a set of outputs. As each state can produce different outputs, we no longer need a separate state for each output combination, and a Mealy FSM can usually be implemented in fewer states.

FSM Example: 2-Bit Clearable Counter

2-bit counter

- 4 states

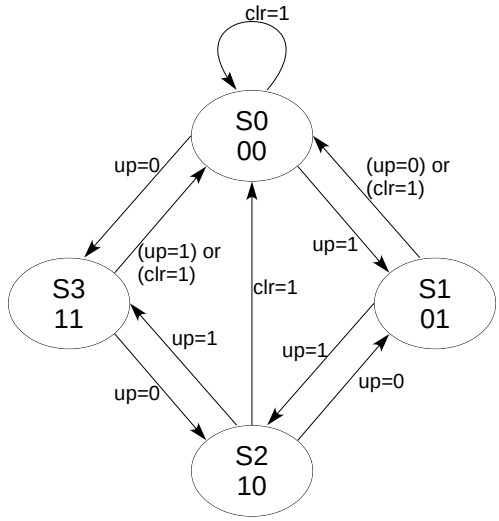
Fully synchronous

Clear count to 00 when `clr=1`

Count up when `up=1`

Count down when `up=0`

Question: What kind of FSM is this?



clr=0 conditions not shown for clarity

When UP=1, states follow in a clockwise direction: 00, 01, 10, 11

When UP = 0, states follow anti-clockwise

All states are connected to S0 (00).

- When CLR =1, S0 is the next state.

By changing the outputs associated with the states, the type of counter can be changed.

- For example, S0=00, S1=01, S2=11, and S3=10 gives a Gray counter.

Only the output is decoding changes.

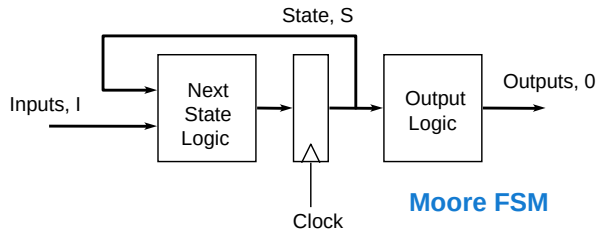
Here is an example of a state diagram for a 2-bit clearable counter. If clr is 1, then we move to state S0 and output 00. While up =1, we move clockwise through the states S1 -> S2 -> S3 -> S0, and the output counts up. When up = 0, we move anticlockwise through the states S3 -> S2 ->S1 ->S0, and the output counts down. Remember, up is sampled, the state changes and the outputs are made on the clock edge. Note that the clr=0 conditions are not shown for clarity. What kind of FSM is this? Since the outputs are only a function of the current state, it is a Moore FSM.

Implementation of an FSM

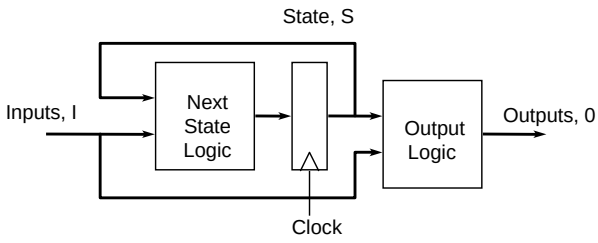
Next state logic decides the next state based on the current state and inputs.

Output logic decodes state (or states and inputs) to produce outputs.

Register stores current state.



Moore FSM



Mealy FSM

For a Moore FSM, the output logic depends only on the state.

For a Mealy FSM, the output logic depends on inputs as well as state.

To implement an FSM, we need three main blocks. The next state logic selects the next state by examining the current state S and the inputs I. On the clock edge, the next state becomes the current state held in the state register. The output decode logic generates the outputs from the current state for a Moore FSM or the state and the inputs for a Mealy FSM.

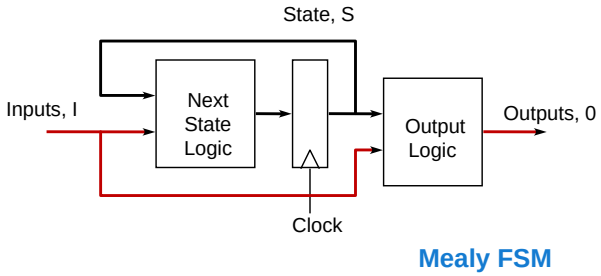
Mealy FSM Outputs

Purely combinational path from inputs to outputs.

- Outputs updated as soon as inputs change.

High probability of glitches on outputs.

- Unsynchronized inputs
  - Input arriving at different times causes intermediate transitions on outputs.
- Glitches on inputs
  - Fed straight through to outputs.
- Different paths in output logic
  - Inputs arrive at the same time, but different paths to output with different timing cause intermediate output transitions.



Mealy FSM

Unsynchronized inputs may be caused by FSM inputs being generated from different logic sources. Different path lengths will introduce differing delays on the inputs, even if all are generated from registered logic with the same clock.

Inputs may be generated by purely combinational logic or even directly from the outputs of a Mealy FSM. Therefore, the inputs may glitch before reaching a steady state, and these glitches can affect the FSM outputs.

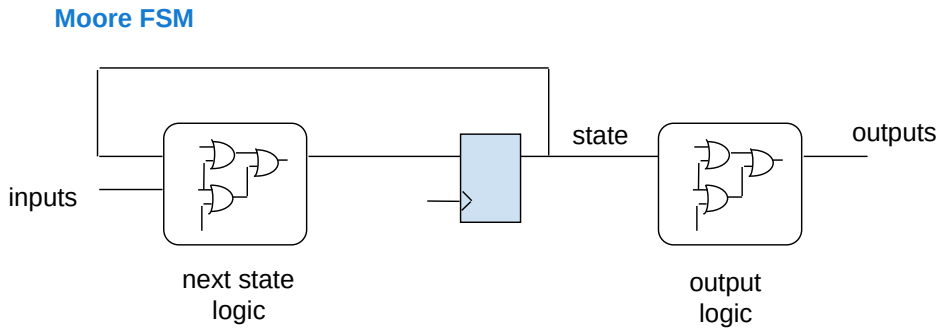
Even if the inputs are synchronized and glitch-free, different logic paths in the output logic may introduce skew between the inputs and hence glitches in the outputs.

The issue with a Mealy FSM is that there is a pure combinational path from the inputs to the outputs. Therefore, changes in the inputs can immediately update the outputs. This gives a high probability of glitches or spurious transitions on the outputs from several possible sources. The inputs may be unsynchronized. If the inputs come from different logic sources, they may not all arrive at the same time, causing the outputs to glitch as they first settle on an intermediate state before being updated by a late arriving input. Even if the inputs are synchronized, any glitches on the inputs are fed straight through to the outputs. Finally, even if the inputs are synchronized and glitch-free, different timing paths in the output logic may introduce skew between the inputs and hence intermediate transitions on the outputs.

FSM Implementations: Combinatorial Outputs

Characteristics

- Simple implementation
- Slower clock to output
- No protection for Mealy outputs



Requires additional logic levels

- Slower clock to output

Let’s look at some standard FSM implementations. The simplest form is a single register block for the state and combinational blocks for the next state logic and outputs. Disadvantages are possible longer delays from clock to output, depending on the size of the output logic, and no protection for Mealy outputs due to the direct combinational path to the output.

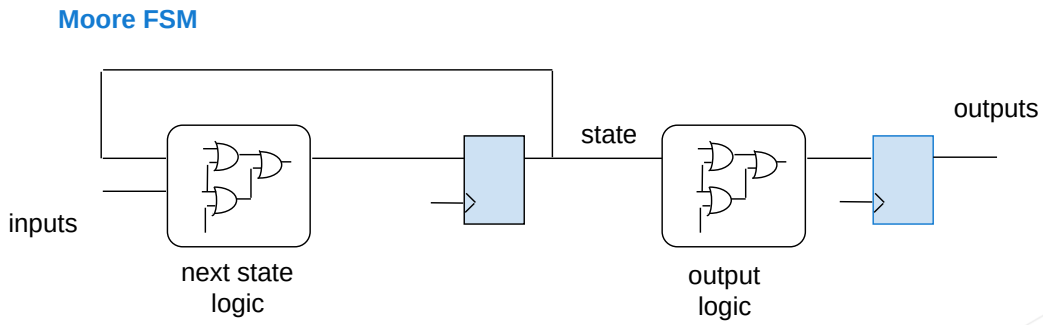
FSM Implementations: Registered Outputs

Requires extra flip-flops for all the outputs.

- Removes glitches on outputs.
- Useful for Mealy machines!

Creates a clock cycle latency on each output.

- Can your design afford it?



10

cadence

Good design practice

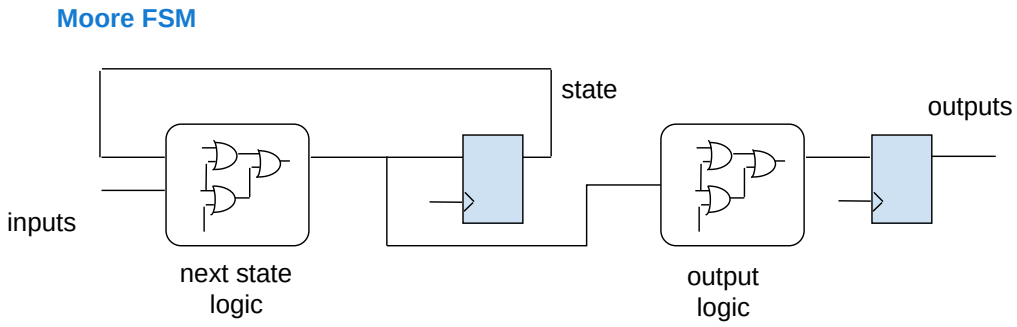
- Isolates time-critical combinational logic paths

A standard option to remove glitches is to register the FSM outputs. This breaks the direct combinational path to outputs for a Mealy machine, and so prevents output glitches. Another advantage is zero delays from clock to output. The disadvantage is an extra clock cycle delay for the output. When the state changes, the outputs for that state are not available until the next clock cycle. This could be a problem for your design.

FSM Implementations: Advanced Registered Outputs

Outputs are decoded from the Next State information.

- Removes the clock cycle latency on the outputs.
- Affects the maximum working frequency.
  - Next state and output decode logic are cascaded.



11

cadence

Affects the max working frequency

- Next state logic and output logic are cascaded
- Some tools can merge the logic

The extra clock cycle latency for registered outputs can be removed by decoding the outputs from the next state logic rather than the current state register. Now a clock loads the next state into the state register and the outputs for that state into the output register on the same cycle. The disadvantage is a longer timing path from inputs to outputs through the cascaded next state and output logic. This will limit the maximum working frequency of the FSM.

State Bits Decoded Outputs

Output values correspond directly to state vector bits.

- Requires careful encoding of the state bits.

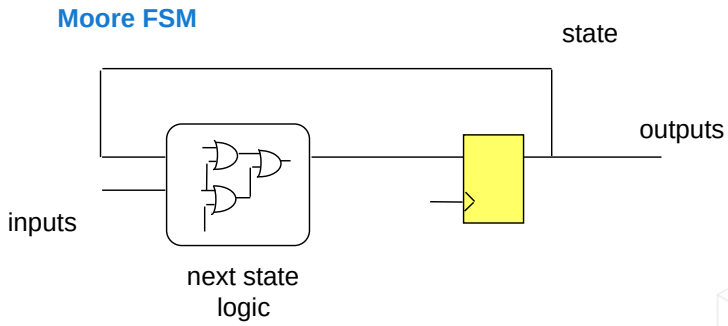
Best implementation for performance.

- Minimizes or removes output logic.

Harder to implement.

- Might not be possible for some designs.

Harder to maintain.



Works better than other solutions

- Good area and performance
- Harder to implement
- Harder to maintain

An ideal solution for Moore machines may be to remove the output decode logic entirely by directly driving the outputs from bits of the state vector. This requires careful encoding of the state vector and may be hard, or even impossible, to implement and hard to maintain. Removing the output logic generates outputs on the right clock cycle without delay and gives the best performance.

Other State Encodings

Possible to choose any pattern of bits to represent state.

- May simplify the next state or output decode logic.

Common choices:

- Gray coding
  - Cyclic code
  - Only one bit changes per change of code
    - Low power and low noise
  - Best for predictable state transition patterns
- One-hot encoding
  - All bits zero except one non-zero bit
  - Good for output decode
  - High register count
    - N states requires an N bit state variable

Binary	Gray	OneHot
000	000	00000001
001	001	00000010
010	011	00000100
011	010	00001000
100	110	00010000
101	111	00100000
110	101	01000000
111	100	10000000

13

cadence

One-hot

- Faster implementation at the expense of the area
  - State vectors are longer

Gray codes

- Prevent intermediate ‘spurious’ states from being generated
- Low power

Modifying the state encoding is a key method for optimizing an FSM by simplifying the next state or output decode logic, reducing spurious state transitions, or reducing power. There are several common choices. As previously mentioned, Gray code is a cyclic code where only one-bit changes from one value to another. This reduces power and noise. Even in an FSM with state-bit encoded outputs, clock skew between the state registers may mean the state bits don’t all change at the same time. So, with binary encoding, switching from a state encoding of 3 to a state encoding of 4 may actually transition through the encoding of 1 and 0 if individual bit changes are not synchronized. As only one-bit changes in Gray code, intermediate encodings are avoided. However, Gray code works best when there is a predictable sequence of state transitions. One-hot encoding is a good alternative. With one-hot encoding, each state is allocated a specific bit, that bit is set for the state, and all other bits are zero. This gives a 2-bit change for any state transition, so minimizing power and noise in arbitrary state sequences. Output decode can also be minimized. The cost is a higher register count since we need a register for every state. So, 8 states need 8 registers with one-hot encoding, compared to 3 registers with binary. In register-rich FPGA technologies, one-hot encoding is the standard.

Complex FSMs

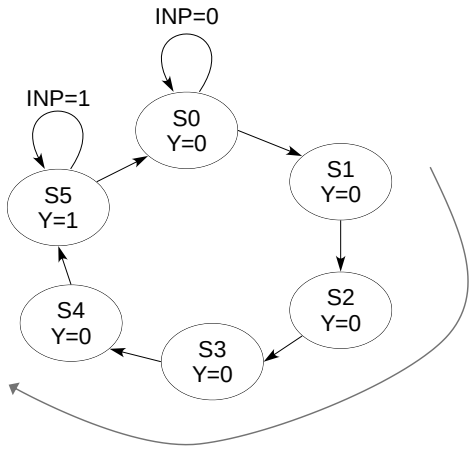
Embed data path elements in FSMs to simplify the design.

- For example, counters to count clock cycles.

The state diagram shows 4 states “delay” before output Y goes high.

- What if the delay changes to a 50-cycle delay?

Simplify state diagram by adding counter . . .



FSM can contain many states that perform relatively mundane functions.

- E.g., waiting for N-clock cycles

Add datapath elements (e.g., counters) to reduce overall complexity.

The example shown adds a delay of 4 clock cycles.

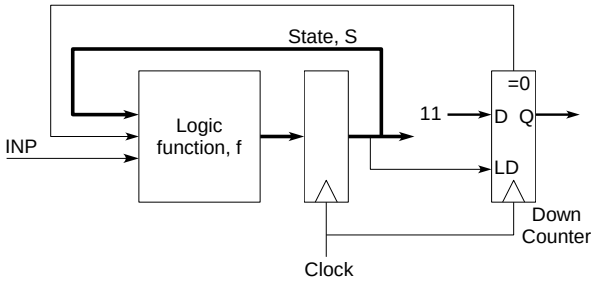
- Could be represented easily as a standard FSM.

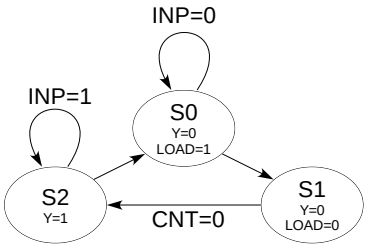
However, if the delay were changed to 100 clock cycles, another 96 states would need to be added.

There are many other techniques to optimize FSMs besides state encoding. For example, it may be possible to simplify a complex FSM by embedding data path elements. The state diagram shows that when INP goes high, states S1 to S4 are unconditional transitions and simply count clock cycles until Y is set. A small number of delay cycles could be implemented with separate states, but what happens if we need a 50-cycle delay? We could use a counter to count cycles and simplify the FSM.

Simplified State Representation

Counter could be used at different places in the state diagram for different purposes.





Counter is held in load state in S0.

When the state changes to S1, the load input is removed, and the counter begins to count down.

When count=0 the FSM moves to S2.

If a delay of 100 was required, the 2-bit counter could simply be extended to 7 bits.

We add an output to the FSM called a load, which is held high in state SO. When we change the state to S1, LOAD is released, and the counter starts counting down cycles. When the count is zero, we transition to S2 and set Y. We could use different load values in the counter at different places in the state diagram for different purposes. The counter reduces the number of states in the state diagram and simplifies the FSM.

Summary

In this module, you learned:

- Finite State Machines are the clearest and simplest way of designing control.
- Supported by a well-defined design process and synthesis.
- Can be augmented by adding datapath elements to process data or to provide explicit support for issues such as delays.

cadence

In summary, Finite State Machines are a clear, simple way of implementing control signals. They are supported by a well-defined design process with standard optimization techniques and dedicated synthesis tools. FSMs can be simplified by adding data path elements such as counters too, for example, process data or implement delays.

Test Your Understanding

- What are the three major functional blocks in an FSM?
- What is one-hot encoding?
- The outputs of a Mealy FSM are a function of what?
- Name a disadvantage and an advantage of using a Mealy FSM over a Moore FSM.

cadence

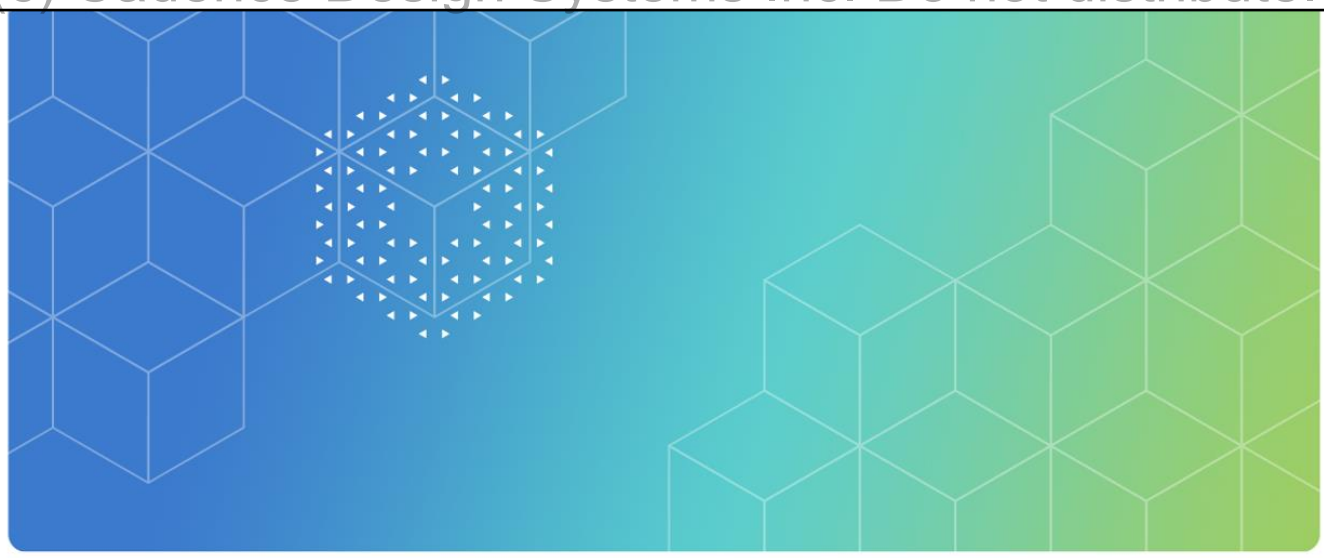
To test what we learned in this module, here is a quick exercise. Pause here, write down your answers, and check against the solutions.

The three major blocks in an FSM are the next state and output decode logic and the state register.

One-hot encoding is a pattern where only one bit of the state vector is set, and all other bits are cleared.

The outputs of a Mealy FSM are a function of the state and the FSM inputs.

An advantage of a Mealy over a Moore is that the Mealy one can usually be implemented in fewer states, as it does not need a separate state for every unique combination of outputs. A disadvantage of Mealy is that the combinational path from inputs to outputs has a high probability of introducing glitches in the FSM outputs.



Module 8.2

Memory Structures

cadence

This page does not contain notes.

Objective

In this module, you will:

- Identify various memory structures and organizations.

Topics include:

- Memory compilers
- Memory organization
  - Synchronous memory
  - Dual ported memory
- LIFO and FIFO
- Modeling memory

In this submodule, we will identify various forms of memory structure and their organization. We will look at memory compilers and memory organization for synchronous and dual-port memories. We will explain LIFO and FIFO blocks and finish by examining memory caches.

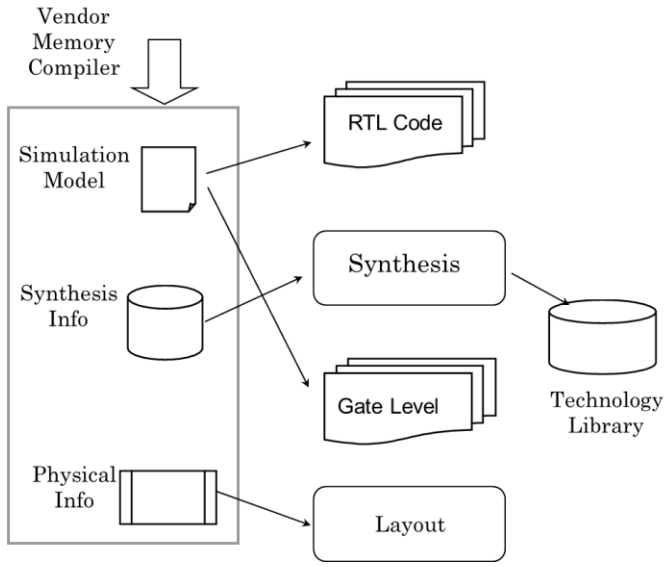
Memory Compilers

Synthesis tools cannot generally infer memory blocks from code.

- Memory must be instantiated as a component.

Models generated by vendor-specific memory compiler.

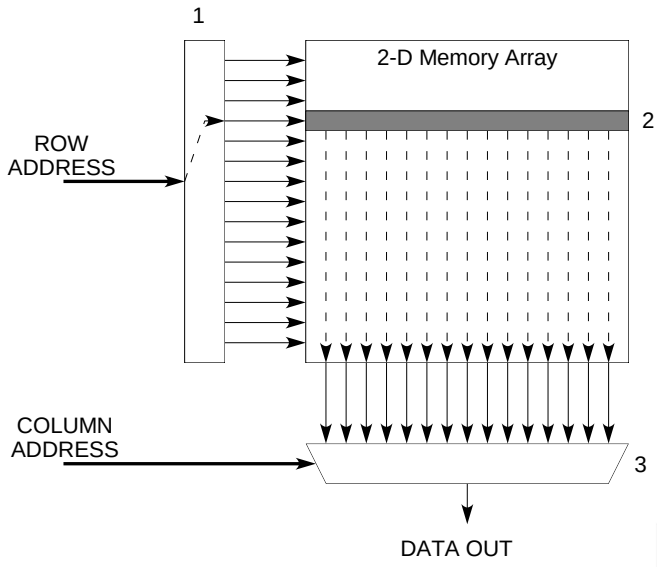
- Not very portable – tied to a vendor.
- Technology must be selected before models can be generated.
- Models will need to be changed for new technology.



Synthesis tools cannot generally synthesize memory blocks directly from HDL code. Memory must be created as a separate component block. For verification and simulation, the memory component is mapped to a high-level behavioral model. For synthesis, we need some memory timing information, but the memory itself is excluded from synthesis. In layout, we replace the memory block with a prebuilt vendor implementation. So, we need memory models for RTL and gate-level simulation, synthesis timing information, and a layout implementation. All these models are generated by a vendor-specific memory compiler. The models are vendor and technology-specific, so you need to select these before generating the models, which limits the portability of your code. If the technology changes, we need new models.

Structural Organization of Memory

- Half of address is decoded to select a whole row of “cells” in the array.
- The data from the row propagates to the edge of the array.
- The other half of the address is used to select an individual datum.



21

cadence

Memory is organized as a 2-dimensional array of data. To access an individual location, half the memory address is decoded to select a row in the memory array. This row of data propagates to the edge of the array, where the other half of the address is decoded to select an individual data item.

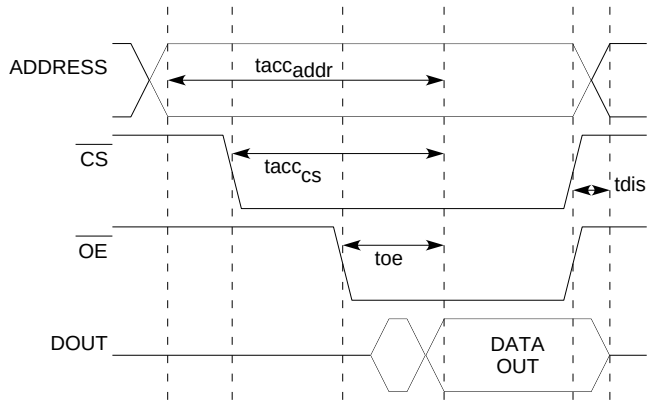
Memory Timing

Accessing is relatively slow

- Memory designed for density, not for performance

For example, read access time is defined for:

- Change of address
- Change of device select (CS)
- Change of output enabling (OE)

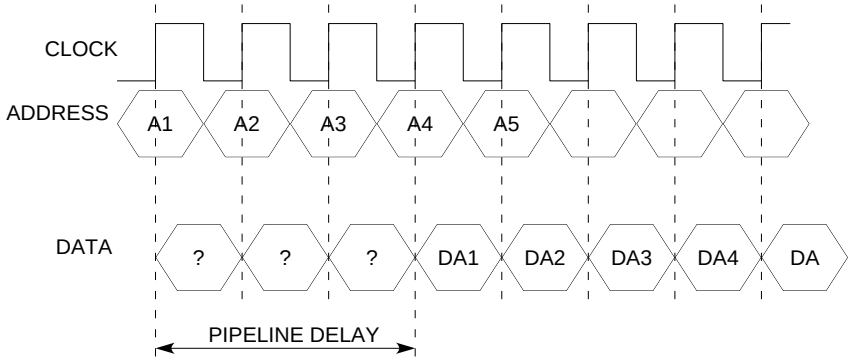


Memories are built for storage density, not for performance, therefore, memory access is relatively slow. Vendor timing diagrams define the timing of various access scenarios. For example, a read operation where the address changes or the device selects, or the output enables. We may need additional logic around a vendor-specific memory to synchronize the vendor-specific timing to the timing used by the design.

Synchronous Memory

Synchronous memories use clocks.

- Synchronizes memory with the associated processor.
- Memory throughput can be improved.
  - Especially for contiguous memory accesses, e.g., cache transfers.
- Introduces “pipeline” delays.



Memory access can be simplified using synchronous memories, where memory operations are synchronized to the system or associated processor clock. This allows memory throughput to be improved, particularly for accessing blocks of data in adjacent addresses. Synchronous memories have internal register banks on address and data buses. The downside is that synchronous memories introduce pipeline delays due to these internal register banks. For example, read data will not be available in the same clock cycle as the address is applied but in a future clock cycle, by which time the address input has moved on. Here the pipeline delay means the data for address A1 is not output until 3 clock cycles after A1 is written to the input, by which time we are applying address A4. Many standard bus protocols use pipelining to take advantage of synchronous memory.

Modeling Synchronous Memory

Memories often have a complex instruction set.

- Internal counters to transfer data in contiguous blocks.

Each stage of an access should be pipelined for the best performance:

- Row decoder
- Array access
- Final data multiplexing

Access time:

- Within a row, one datum per clock cycle after latency.
- Between rows, latency can be hidden.

cadence

Synchronous memories often have complex instruction sets. For example, to allow burst mode operations that access memory in blocks of 8 or 16 addresses. A burst mode access may provide a single address, and internal counters automatically access a set number of locations from the address. For the best performance, every part of the memory organization, row access, data propagation, and output data MUXing can be pipelined with register banks. By storing a whole row of data in an output register bank, we can easily read a block of data from the row by simple indexing. A synchronous memory may also preload data from adjacent rows into separate output register banks in anticipation that the addresses of a memory access will spread over several rows. This hides the normal row access delay due to data propagation.

Dual Port Memory

Memory can be designed to support two separate ports:

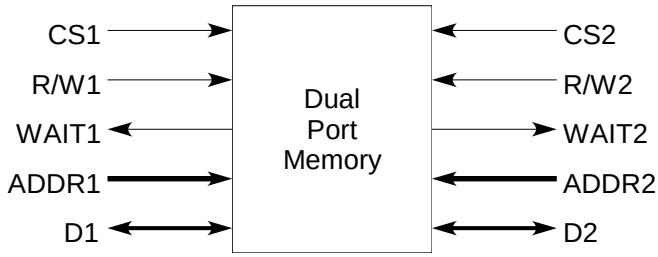
- Inherently at the transistor level...
- ...or “Time-sliced” access to a normal memory array.

Ports may be different:

- Both read/write.
- One write, one read.

Control of access is important.

- Simultaneous access to a single location from both ports can give indeterminate behavior.



25

cadence

In some designs, dual port memory is more useful than a single port. Dual port memory has two separate sets of access signals, such as address, data, read/write, etc. Support for the two ports can be added to the memory at the transistor level, or we can time-slice the two access ports to a normal single port array. Both ports don't have to be read/write (although that is common). A dual port implementation is simpler if the design needs one port to be write-only and the other read-only. The key issue in implementation is to prevent simultaneous access to a single location from both ports. Simultaneous access can lead to indeterminate behavior due to race conditions. For example, the result of writes from both ports to the same location depends on which write completes first. Access may be easier to control in a time-sliced normal memory rather than a transistor-level implementation.

Stack: Last-In-First-Out Memory

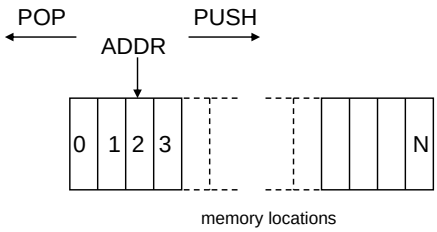
Implemented with an up/down counter.

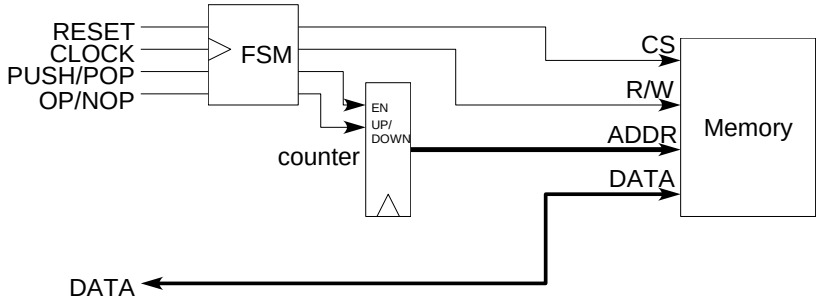
PUSH

- Increment counter then write.

POP

- Read then decrement counter.





Another useful memory-based component is a stack, or a Last-In-First-Out (LIFO) memory. Data written to a stack is stored in order of writing, and a read accesses the last data written. With a memory-based stack, the memory generates the address using a counter which can be incremented or decremented. A stack write is called a push, and this increments the counter and writes to the memory at the counter address. A stack read, or pop, reads the memory at the current counter address and then decrements the count. So, the counter provides the address to the top of the stacked data. We need limits on the counter to prevent overflow at the end of the address space and to detect error conditions like a pop from an empty stack (count = 0) or a push to a full stack (count = max).

First-In-First-Out Memory (FIFO)

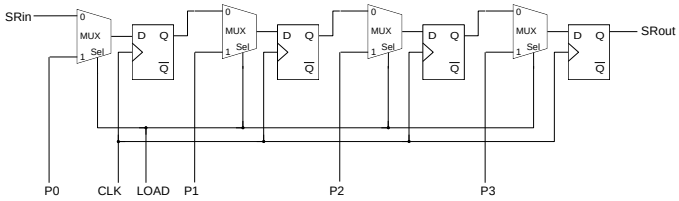
Used as delay, queue, or “rate buffer.”

- Data written in sequentially at tail of queue.
- Data read sequentially from head of queue.

Implemented as memory or shift register.

Shift Register FIFO can suffer from “fall through.”

- Time taken for the data to propagate through the FIFO to the end of the queue.
- Can be used as a delay element.
  - Fixed time (clock cycles) to pass through the FIFO.



Parallel In, Serial In, Serial Out (PISISO) shift register

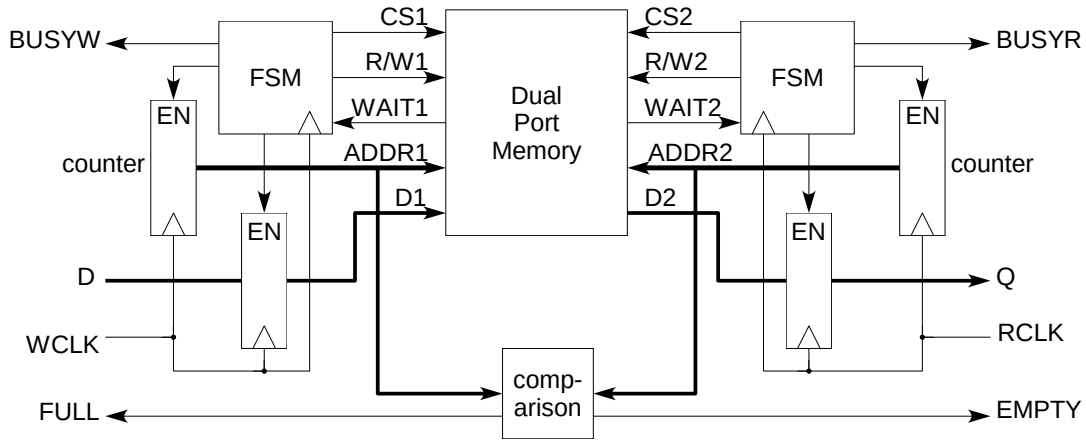
A variant of the stack is a First-In-First-Out (FIFO) memory. Also called a delay, queue, or rate buffer. Data is still stored in a FIFO in write order, but in a FIFO, we write data to the tail of a data queue and read data from the head. So, data is read in the order in which it was written.

A FIFO could be implemented using a memory block or a shift register, a block of serially connected registers. A simple shift-register FIFO only passes data on a clock edge; therefore, for example, if you push data to a shift register FIFO of depth 8, it will take 8 clock cycles for the data to propagate through the registers before it can be popped from the output. This is called fall through time. The shift register FIFO is excellent as a delay element, where you want to store data for a fixed number of clock cycles before processing it.

A simple shift register could mitigate fall-through time by adding a parallel load facility. The example here can be loaded via the serial SRin, or in parallel using P0-P3. The output is serial only via Srout, making this example a Parallel In, Serial In, Serial Out (PISISO) shift register.

Synchronous FIFO (Dual Clock)

Based around dual-port memory.



A memory-based FIFO can be constructed around a dual port memory, with one write port and one read port. Address generation is complicated, as we need separate tails to write and header-read addresses. So, we have separate address counters for push and pop. The addresses follow each other through the memory address space, with the push address leading the pop. The addresses must be able to wrap round from maximum to minimum. We can determine the status of the FIFO by comparing the addresses. For example, if the write address points to the next free location (i.e., write then increment), then if the read is one less than the write address, the FIFO is empty. If the read and write addresses are the same, the FIFO is full. To avoid losing data, a full FIFO should not be written, and an empty FIFO should not be read. The FIFO may need additional logic and full and empty status outputs to prevent this.

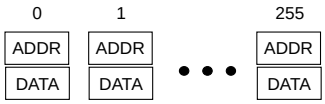
Static Cache Memory Modeling

Only a small percentage of memory may be used.

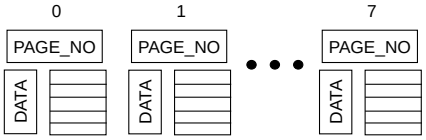
- Can be modeled as element or paged cache:
  - Element caches implement a fixed number of individual memory locations.
  - Paged cache implements fixed-size data blocks.

Maximum number of memory locations used must be known.

64Kx16 RAM as 256 element cache



64Kx16 memory as 8X4K paged cache



29

cadence

If memory is sparsely used, design memory models as caches to only implement the number of memory locations required.

Element caches implement a fixed number of individual memory locations.

- On read or write, use a loop to read through the addresses until the required location is found.

Paged cache implements fixed-size data blocks.

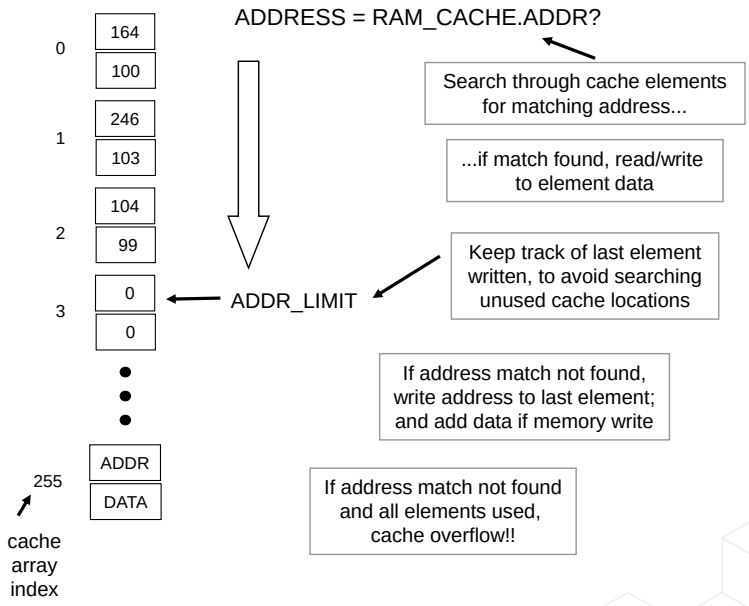
- On read or write, use a loop to read through the addresses until the required address range is found, and index data block to access the required location.

Some designs may only use a small percentage of the available memory space, and addresses used may not be contiguous. Instead of implementing the whole memory, a design could implement only the locations used as a cache. An element cache implements individual memory locations. The address and data are stored together as one entry in the cache. It may be more efficient to create a paged cache that stores fixed-size blocks of data with a page number indicating an address range and the individual address-data pairs stored in the page block. Caches have a fixed size which limits the number of elements or pages that can be stored, so cache management is a necessity. Caches are also used in processor designs to implement fast, local storage of frequently used memory locations, to reduce main memory access delays.

Element Cache Model (Static)

Questions that an implementation will need to address:

- What should happen on a cache overflow?
- What should happen for a read on an un-cached address:
  - Do we create a cache location?
  - What is the output data value?



ADDR_LIMIT keeps track of the last cache array element written and saves time searching unused locations.

With the new address input, search from the first location to ADDR_LIMIT to find a matching address.

- If found, read or write to or from an element data field.
- If not found, then If ADDR_LIMIT <= maximum cache size, write address to ADDR_LIMIT location and write data.

What should we do for a memory read to an unwritten address?

- If ADDR_LIMIT > max. cache size, we have cache overflow, and the data can be lost.

We need to know the maximum cache size in advance or use a method of dynamically growing/shrinking the cache size during simulation.

In summary, there are many different memory architectures for different design applications. Memory cannot generally be synthesized, so we need a vendor-specific memory compiler to provide the simulation, synthesis, and place and route models for the design. We can add additional logic around basic single or dual port memories to implement stack or FIFO structures.

Summary

In this module, you learned:

- There are different memory architectures that can be exploited in different applications.
- Memory cannot generally be directly synthesized.
- Basic memory blocks can be generated efficiently by compilers.
- Interface logic can be added to basic single/dual port memories to build more complex memory structures:
  - FIFO
  - Stack

cadence

This page does not contain notes.

Test Your Understanding

- What are the three stages of memory access as described in the notes?
- What is “fall-through” time?
- Name three applications of a FIFO.

cadence

To test what we learned in this module, here is a quick exercise. Pause here, write down your answers, and check against the solutions.

The three stages of memory are address decode for row selection, row data propagation to the memory edge, and address decode for the data output selection. Fall-through time is the delay in a shift-register FIFO for the input data to be clocked through the register array to the outputs. Applications of a FIFO include rate-buffer, queue and delay.